CSA08 – PYTHON PROGRAMMING

Assignment 3 – CO3

Lists, Tuples, Dictionaries, Sets and List Comprehension

Field	Details
Name	S.Harish Raghav
Register number	1912260556
Python version	Python
GitHub repository	GitHub - harishraghavsk-dot/python-assesment · GitHub

Contents and repository layout

Every question below follows the same order: problem → implementation (code screenshots) → output screenshots → design and analysis → experimental investigation → conclusion → alternative concept and complexity analysis. All timings were measured with `time.perf_counter()` (best of several runs) on one machine, so absolute values will differ on yours; the growth patterns are what matter.

```
CSA08-<RegisterNumber>-Assignment3/
  README.md   requirements.txt
  Task01/q1_rainfall.py       output.txt  screenshots/
  Task02/q2_tuples.py         output.txt  screenshots/
  Task03/q3_dicts.py          output.txt  screenshots/
  Task04/q4_sets.py           output.txt  screenshots/
  Task05/q5_comprehension.py  output.txt  screenshots/
  Task06/q6_timetable.py      output.txt  screenshots/
  Task07/q7_contacts.py       output.txt  screenshots/
```

Q1 – Lists and Slicing: Daily Rainfall Records (6 marks, K3 – Apply)

1.1 Implementation

Readings are stored in one list of 365 floats (`0` = dry day). Weekly totals use slicing (`r[i:i+7]` for `i` in steps of 7), the extremes use `max`/`min` with a `key` so the **position** is also returned, the moving average is computed three ways (naive slices, running sum, `deque`) and the dry days come from `enumerate`. The last (53rd) week has only one day because 365 = 52 × 7 + 1.

q1_rainfall.py (lines 1-42)

```
"""Q1 - Lists and slicing: Daily rainfall records"""
import random, time
from collections import deque

random.seed(1)

def weekly_totals(r):
    return [sum(r[i:i + 7]) for i in range(0, len(r), 7)]

def max_min(r):
    if not r:
        return None
    hi = max(range(len(r)), key=lambda i: r[i])
    lo = min(range(len(r)), key=lambda i: r[i])
    return (hi, r[hi]), (lo, r[lo])

def ma_naive(r, w=7):
    """re-adds w values for every window"""
    if len(r) < w:
        return []
    return [sum(r[i:i + w]) / w for i in range(len(r) - w + 1)]

def ma_running(r, w=7):
    """running sum: one add and one subtract per step"""
    if len(r) < w:
        return []
    s = sum(r[:w])
    out = [s / w]
    for i in range(w, len(r)):
        s += r[i] - r[i - w]
        out.append(s / w)
    return out

def ma_deque(r, w=7):
    d, s, out = deque(maxlen=w), 0, []
    for x in r:
        if len(d) == w:
            s -= d[0]
        d.append(x); s += x
        if len(d) == w:
            out.append(s / w)
    return out
```

Screenshot Q1 source code (1 of 3)

q1_rainfall.py (lines 43-84)

```
 1
 2 def gen(n):
 3     return [0 if random.random() < 0.6 else round(random.uniform(0.5, 60), 1) for _ in
       range(n)]
 4
 5 readings = gen(365)
 6 print("=== Implementation (365 days) ===")
 7 wk = weekly_totals(readings)
 8 print("Number of weeks (last is partial):", len(wk))
 9 print("First 5 weekly totals:", [round(x, 1) for x in wk[:5]])
10 full = wk[:52]
11 print("Wettest week (1-based):", full.index(max(full)) + 1, "total", round(max(full),
       1))
12 print("Driest  week (1-based):", full.index(min(full)) + 1, "total", round(min(full),
       1))
13 (hi, hv), (lo, lv) = max_min(readings)
14 print(f"Max reading {hv} mm on day index {hi}; Min reading {lv} mm first at index {lo}")
15 ma = ma_running(readings)
16 print("Moving-average length:", len(ma), "first 3:", [round(x, 2) for x in ma[:3]])
17 dry = [i for i, x in enumerate(readings) if x == 0]
18 print("Dry days:", len(dry), "first 10 indices:", dry[:10])
19
20 print("\n=== Slicing demo on r = list(range(1,22)) ===")
21 r = list(range(1, 22))
22 print("r[7:14]  ->", r[7:14])
23 print("r[-7:]   ->", r[-7:])
24 print("r[::7]   ->", r[::7])
25
26 print("\n=== Verify the three moving averages agree ===")
27 a, b, c = ma_naive(readings), ma_running(readings), ma_deque(readings)
28 ok = all(abs(x - y) < 1e-9 for x, y in zip(a, b)) and all(abs(x - y) < 1e-9 for x, y in
       zip(b, c))
29 print("identical:", ok, "| lengths:", len(a), len(b), len(c))
30
31 print("\n=== Edge cases ===")
32 print("empty list:", weekly_totals([]), max_min([]), ma_naive([]), ma_running([]))
33 print("3 values  :", weekly_totals([1, 2, 3]), ma_naive([1, 2, 3]), ma_running([1, 2,
       3]))
34
35 print("\n=== Additions count (window w=7) ===")
36 for n in (365, 36500, 365000):
37     naive_adds = 6 * (n - 6)
38     run_ops = 6 + 2 * (n - 7)
39     print(f"n={n:>7}: naive adds={naive_adds:>8}  running-sum ops={run_ops:>8}
         ratio={naive_adds/run_ops:.2f}")
40
41 print("\n=== Timing (seconds, best of 3) ===")
42 print(f"{'n':>8} {'naive':>10} {'running':>10} {'deque':>10}")
```

Screenshot Q1 source code (2 of 3)

q1_rainfall.py (lines 85-106)

```python
for n in (365, 36500, 365000):
    d = gen(n)
    res = []
    for f in (ma_naive, ma_running, ma_deque):
        times = []
        for _ in range(3):
            t0 = time.perf_counter(); f(d); times.append(time.perf_counter() - t0)
        res.append(min(times))
    print(f"{n:>8} {res[0]:>10.5f} {res[1]:>10.5f} {res[2]:>10.5f}")

print("\n=== Late reading: list.insert cost ===")
for n in (36500, 365000):
    d = gen(n)
    t0 = time.perf_counter()
    for _ in range(200):
        d.insert(0, 1.0)
    t_front = (time.perf_counter() - t0) / 200
    t0 = time.perf_counter()
    for _ in range(200):
        d.append(1.0)
    t_end = (time.perf_counter() - t0) / 200
    print(f"n={n}: insert(0) {t_front*1e6:.2f} us, append {t_end*1e6:.2f} us")
```

Screenshot Q1 source code (3 of 3)

$ python q1_rainfall.py (output part 1)

```
=== Implementation (365 days) ===
Number of weeks (last is partial): 53
First 5 weekly totals: [93.3, 95.0, 23.2, 0, 139.7]
Wettest week (1-based): 47 total 208.7
Driest  week (1-based): 4 total 0
Max reading 59.1 mm on day index 250; Min reading 0 mm first at index 0
Moving-average length: 359 first 3: [13.33, 13.33, 10.51]
Dry days: 231 first 10 indices: [0, 2, 3, 4, 6, 7, 10, 13, 14, 15]

=== Slicing demo on r = list(range(1,22)) ===
r[7:14]  -> [8, 9, 10, 11, 12, 13, 14]
r[-7:]   -> [15, 16, 17, 18, 19, 20, 21]
r[::7]   -> [1, 8, 15]

=== Verify the three moving averages agree ===
identical: True | lengths: 359 359 359

=== Edge cases ===
empty list: [] None [] []
3 values  : [6] [] []

=== Additions count (window w=7) ===
n=    365: naive adds=    2154  running-sum ops=     722  ratio=2.98
n=  36500: naive adds=  218964  running-sum ops=   72992  ratio=3.00
n= 365000: naive adds= 2189964  running-sum ops=  729992  ratio=3.00

=== Timing (seconds, best of 3) ===
       n      naive    running      deque
     365    0.00009    0.00003    0.00005
   36500    0.00973    0.00349    0.00462
  365000    0.10081    0.04090    0.05064

=== Late reading: list.insert cost ===
n=36500: insert(0) 6.93 us, append 0.04 us
n=365000: insert(0) 115.57 us, append 0.05 us
```

Screenshot Q1 execution and output (1 of 1)

1.2 Design and analysis

Pseudocode – 7-day moving average (running sum)

```
FUNCTION moving_average(r, w = 7)
    IF length(r) < w THEN RETURN empty list
    s   <- r[0] + r[1] + ... + r[w-1]          // w-1 additions, once
    out <- [ s / w ]
    FOR i FROM w TO length(r) - 1
        s <- s + r[i] - r[i - w]               // add newest day, drop oldest day
        APPEND s / w TO out
    RETURN out
```

What the slices return

- `readings[7:14]` – a **new list** with the items at index 7 to 13, i.e. the second week (days 8–14). The end index is excluded, so it has 14 − 7 = 7 items.
- `readings[-7:]` – the **last seven** items (the final week of the data). Negative indices count from the end.
- `readings[::7]` – every 7th item starting at index 0: the first day of every week (53 items for 365 values). In the demo on `list(range(1,22))` they return `[8..14]`, `[15..21]` and `[1, 8, 15]`.

Why a list, and what the operations cost

A year of readings is an ordered, indexed, mostly append-only sequence where position = day. A list gives O(1) indexing, cheap slicing and in-place updates. Costs (n = current length): `append` amortised **O(1)**; `pop()` **O(1)**; `insert(i, x)` and `pop(i)` **O(n − i)** because later items are shifted; `sort()` **O(n log n)** (Timsort, O(n) if already sorted).

Number of additions for the moving average (window w = 7)

- **Naive:** every window adds 7 values = 6 additions, and there are n − 6 windows → **6(n − 6)** additions.
- **Running sum:** 6 additions for the first window, then one addition and one subtraction per step → 6 + 2(n − 7) operations. For a fixed window both are O(n); the naive version is O(w·n) and the running sum O(n), so the running sum is about (w − 1)/2 = 3 times cheaper for w = 7.

n	Naive additions 6(n−6)	Running-sum operations	Ratio
365	2,154	722	2.98
36,500	218,964	72,992	3.00
365,000	2,189,964	729,992	3.00

1.3 Experimental investigation

Readings were generated with `random` (about 60 % dry days). All three moving averages gave identical results (`identical: True`). Edge cases: the empty list returns `[]` / `None` without error, and a list with fewer than 7 values gives an empty moving average (no complete window exists) while the weekly total still works.

n (days)	Naive (s)	Running sum (s)	deque (s)
365	0.00009	0.00003	0.00005
36500	0.00973	0.00349	0.00462
365000	0.10081	0.04090	0.05064

The running sum is consistently about 2–3× faster than the naive method, matching the derived ratio of 3 (slightly lower because the naive version's inner `sum()` runs in C while the running-sum loop runs in Python bytecode). Time per element is constant, so both grow linearly with n: ×100 more data costs ~×100 more time. The `deque` version is also O(n) but pays method-call overhead.

1.4 Design question – a late reading

n	insert(0) (µs)	append (µs)
36,500	6.93	0.04
365,000	115.57	0.05

`list.insert()` is **O(n)**: it moves every later element one slot right (≈ 7 µs for 36,500 items and ≈ 100+ µs for 365,000, against well under 0.1 µs for `append`). For one year (365 items) that is a few microseconds, so **yes, I would still use a list**. Better still: because the index equals the day number, pre-create 365 slots (`[None]*365`) and assign `readings[day] = value`, which is O(1) and needs no shifting. If late, out-of-order arrivals were frequent on huge data, a dictionary keyed by date (or a database) would be the better choice.

1.5 Alternative concept and complexity analysis

- **`collections.deque(maxlen=7)`:** keep only the current window. `append` is O(1) and the oldest value falls out automatically; keep a running total (subtract `d[0]` before appending). Time O(n) for n days, extra space **O(7) = O(1)** besides the output (O(n − 6) averages).
- **Dictionary keyed by date:** `rain[date]`, `rain[date - 1 day]`, … gives O(1) average access and handles late or missing days naturally, but each window needs 7 lookups (O(7n), or O(n) with a running sum) and the hash table uses more memory (O(n) with a larger constant than a list) and loses slicing.
- **List slicing (original):** each window creates a temporary 7-item list: O(7n) time, O(7) temporary space per step, O(n) in total for the stored readings.

Approach	Time for n days	Extra space (excluding output)	Notes
List + slices (naive)	O(7n)	O(7) temporary per window	Simplest; constant factor 3× higher
List + running sum	O(n)	O(1)	Fastest in measurements
deque(maxlen=7)	O(n)	O(7) = O(1)	Streaming; no need to keep all days
dict keyed by date	O(7n) / O(n)	O(n) hash table	Handles late/missing dates in O(1)

Q2 – Tuples, Mutability and Copying: City Bus-Stop Map (7 marks, K4 – Analyse)

2.1 Implementation

Stops are `(latitude, longitude)` tuples used as dictionary keys; the zone grid is built with a list comprehension; aliasing, `copy.copy`, slicing and `copy.deepcopy` are compared; tuples are unpacked in a `for` loop. **Complete the Code – TODO 1** is solved by `good = [[0] * cols for _ in range(rows)]`, which runs the expression `[0] * cols` once per row and therefore creates a separate list for every row.

```
# TODO 1 solution
good = [[0] * cols for _ in range(rows)]
```

q2_tuples.py (lines 1-42)

```
"""Q2 - Tuples, mutability and copying: City bus-stop map"""
import copy, sys, time

print("=== Complete the Code (TODO 1) ===")
rows, cols = 3, 4
bad = [[0] * cols] * rows          # every row is the SAME list object
bad[0][1] = 7
good = [[0] * cols for _ in range(rows)]   # TODO 1 solution: separate list per row
good[0][1] = 7
print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])
stops = {(13.0827, 80.2707): "Chennai Central", (13.0418, 80.2341): "T. Nagar"}
print(stops[(13.0418, 80.2341)])

print("\n=== Using a list as a dictionary key ===")
try:
    stops[[13.0418, 80.2341]] = "T. Nagar"
except TypeError as e:
    print("TypeError:", e)

print("\n=== Bug and fix with is / id() ===")
print("bad  row ids :", [id(r) for r in bad])
print("good row ids :", [id(r) for r in good])
print("bad[0] is bad[2] ->", bad[0] is bad[2], "| good[0] is good[2] ->", good[0] is
      good[2])

print("\n=== Aliasing, shallow copy, deep copy ===")
orig = [[1, 2], [3, 4]]
alias = orig
shallow1 = copy.copy(orig)
shallow2 = orig[:]
deep = copy.deepcopy(orig)
orig[0][0] = 99      # change an inner list
orig.append([5, 6])  # change the outer list
print("orig    :", orig)
print("alias   :", alias, "(same object:", alias is orig, ")")
print("copy.copy:", shallow1, "(inner shared:", shallow1[0] is orig[0], ")")
print("slice[:] :", shallow2, "(inner shared:", shallow2[0] is orig[0], ")")
print("deepcopy :", deep, "(inner shared:", deep[0] is orig[0], ")")

print("\n=== Tuple unpacking in a for loop ===")
for (lat, lon), name in stops.items():
```

Screenshot Q2 source code (1 of 3)

q2_tuples.py (lines 43-84)

```
    print(f"{name:16s} lat={lat} lon={lon}")

print("\n=== Function changes caller's list ===")
def add_zone(z):
    z.append("Zone-X")
def add_zone_safe(z):
    z = z + ["Zone-Y"]
    return z
zones = ["A", "B"]
add_zone(zones); print("after add_zone      :", zones)
new = add_zone_safe(zones); print("after add_zone_safe :", zones, "| returned:", new)

print("\n=== Memory: list vs tuple of 1,000,000 values ===")
L = list(range(1_000_000)); T = tuple(L)
print("sys.getsizeof(list) :", sys.getsizeof(L), "bytes")
print("sys.getsizeof(tuple):", sys.getsizeof(T), "bytes")

print("\n=== Timing (seconds, best of 5) ===")
def best(f, n=5):
    ts = []
    for _ in range(n):
        t0 = time.perf_counter(); f(); ts.append(time.perf_counter() - t0)
    return min(ts)
print(f"create list : {best(lambda: list(range(1_000_000))):.5f}")
print(f"create tuple: {best(lambda: tuple(range(1_000_000))):.5f}")
def idx(c):
    def f():
        for i in range(0, 1_000_000, 1):
            c[i]
    return f
print(f"index list  (1e6 reads): {best(idx(L), 3):.5f}")
print(f"index tuple (1e6 reads): {best(idx(T), 3):.5f}")

print("\n=== Changing a list stored inside a tuple ===")
t = ([1, 2], 3)
t[0].append(99)
print("after t[0].append(99):", t, "-> allowed (the tuple still holds the same list)")
try:
    t[0] = [0]
except TypeError as e:
    print("t[0] = [0] ->", "TypeError:", e)
try:
```

Screenshot Q2 source code (2 of 3)

q2_tuples.py (lines 85-104)

```
    t[0] += [5]
except TypeError as e:
    print("t[0] += [5] -> TypeError:", e)
print("but the list was still extended:", t)
try:
    hash(t)
except TypeError as e:
    print("hash(t) ->", "TypeError:", e)

print("\n=== Alternative: dict keyed by (row, col) for non-empty zones ===")
sparse = {}
def board(r, c):
    sparse[(r, c)] = sparse.get((r, c), 0) + 1
for rc in [(0, 1), (0, 1), (2, 3)]:
    board(*rc)
print(sparse, "| zone (0,1):", sparse.get((0, 1), 0), "| zone (1,1):", sparse.get((1,
      1), 0))
big = 1000
grid = [[0] * big for _ in range(big)]
print("1000x1000 list grid outer+rows bytes ~", sys.getsizeof(grid) +
      sum(sys.getsizeof(r) for r in grid))
print("sparse dict (2 non-empty zones) bytes =", sys.getsizeof(sparse))
```

Screenshot Q2 source code (3 of 3)

$ python q2_tuples.py (output part 1)

```
=== Complete the Code (TODO 1) ===
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar

=== Using a list as a dictionary key ===
TypeError: unhashable type: 'list'

=== Bug and fix with is / id() ===
bad  row ids : [140189899770688, 140189899770688, 140189899770688]
good row ids : [140189895653312, 140189895654784, 140189895654848]
bad[0] is bad[2] -> True | good[0] is good[2] -> False

=== Aliasing, shallow copy, deep copy ===
orig    : [[99, 2], [3, 4], [5, 6]]
alias   : [[99, 2], [3, 4], [5, 6]] (same object: True )
copy.copy: [[99, 2], [3, 4]] (inner shared: True )
slice[:] : [[99, 2], [3, 4]] (inner shared: True )
deepcopy : [[1, 2], [3, 4]] (inner shared: False )

=== Tuple unpacking in a for loop ===
Chennai Central  lat=13.0827 lon=80.2707
T. Nagar         lat=13.0418 lon=80.2341

=== Function changes caller's list ===
after add_zone      : ['A', 'B', 'Zone-X']
after add_zone_safe : ['A', 'B', 'Zone-X'] | returned: ['A', 'B', 'Zone-X', 'Zone-Y']

=== Memory: list vs tuple of 1,000,000 values ===
sys.getsizeof(list) : 8000056 bytes
sys.getsizeof(tuple): 8000040 bytes

=== Timing (seconds, best of 5) ===
create list : 0.02603
create tuple: 0.02517
```

Screenshot Q2 execution and output (1 of 2)

```
$ python q2_tuples.py   (output part 2)

index list  (1e6 reads): 0.01942
index tuple (1e6 reads): 0.01849

=== Changing a list stored inside a tuple ===
after t[0].append(99): ([1, 2, 99], 3) -> allowed (the tuple still holds the same list)
t[0] = [0] -> TypeError: 'tuple' object does not support item assignment
t[0] += [5] -> TypeError: 'tuple' object does not support item assignment
but the list was still extended: ([1, 2, 99, 5], 3)
hash(t) -> TypeError: unhashable type: 'list'

=== Alternative: dict keyed by (row, col) for non-empty zones ===
{(0, 1): 2, (2, 3): 1} | zone (0,1): 2 | zone (1,1): 0
1000x1000 list grid outer+rows bytes ~ 8064856
sparse dict (2 non-empty zones) bytes = 224
```

Screenshot Q2 execution and output (2 of 2)

The first four lines of the output match the expected output exactly (`bad` shows 7 in every row, `good` only in row 0, `True False`, `T. Nagar`).

Using a list as a key: `stops[[13.0418, 80.2341]] = ...` raises `TypeError: unhashable type: 'list'`. A dictionary must compute `hash(key)` to place the key, and that hash must never change while the key is stored. A list can be modified, so its contents (and any hash based on them) could change and the entry could no longer be found; Python therefore sets `list.__hash__ = None`.

2.2 Design and analysis

Why tuples can be keys but lists cannot

Tuples are immutable, so their hash (computed from their items) is stable for the lifetime of the object, provided every item is itself hashable. Lists are mutable and unhashable. A tuple that contains a list is also unhashable (shown in the output: `hash(t)` raises `TypeError`).

Memory diagram

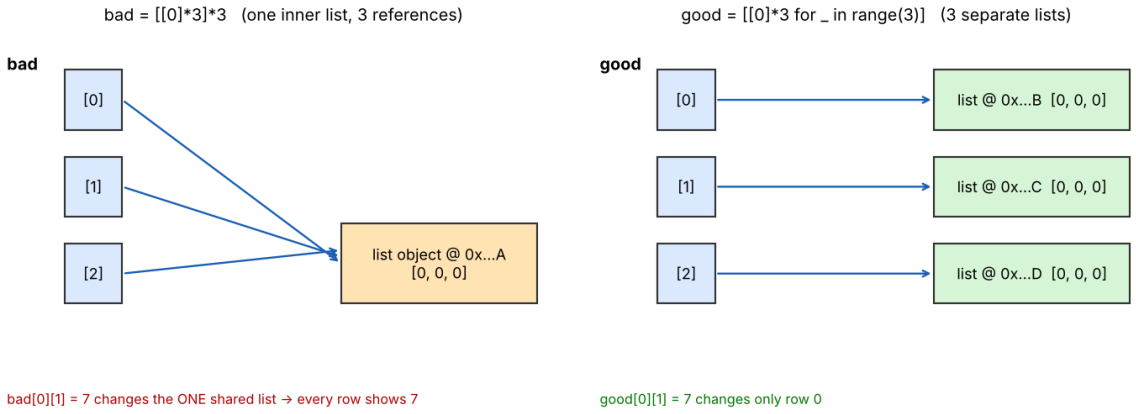


Figure Q2.1 – `[[0]*3]*3` stores three references to ONE inner list; the comprehension creates three lists

Shallow copy versus deep copy

- A **shallow copy** (`copy.copy(x)`, `x[:]`, `list(x)`) creates a new outer container but shares the inner objects. It is enough when the items are immutable (numbers, strings, tuples), i.e. for a flat list.
- A **deep copy** (`copy.deepcopy`) copies the inner objects recursively. It is needed for nested mutable data (a grid, a list of dictionaries) if the copy must change independently. In the

output, after `orig[0][0] = 99`, the alias, `copy.copy` and slice copies all show 99 (`inner shared: True`), whereas the deep copy still shows 1.

Why a function changes the caller's list

Python passes **references to objects** (call by assignment). Inside `add_zone(z)` the parameter `z` is a second name for the same list object, so `z.append(...)` mutates the object the caller sees. Rebinding (`z = z + [...]`) creates a new list and leaves the caller's list unchanged, as `add_zone_safe` shows. To protect the caller, copy first (`z = list(z)`) or return a new list.

2.3 Experimental investigation

- **Bug and fix with `is`/`id()`:** all three row ids of `bad` are identical (`bad[0] is bad[2]` → `True`) while the three rows of `good` have different ids (`False`).

Measurement (1,000,000 values)	List	Tuple
sys.getsizeof (bytes)	8,000,056	8,000,040
Create (s, best of 5)	0.02603	0.02517
Index all 1,000,000 items (s)	0.01942	0.01849

Both containers use about 8 MB (8 bytes per pointer); the tuple is only 16 bytes smaller because it has no spare capacity. `getsizeof` counts the container only, not the integer objects it refers to. Creation and indexing times are essentially equal, so choose tuples for **safety and meaning, not speed**.

- **A list stored inside a tuple:** `t = ([1, 2], 3)`; `t[0].append(99)` **works** because the tuple only stores a reference and the referenced list is mutable. `t[0] = [0]` raises `TypeError` (the tuple's slot cannot be rebound). `t[0] += [5]` raises `TypeError` too, **but the list is still extended** (`+=` first calls `list.__iadd__`, which mutates in place, and only then tries to store the result back into the tuple). Immutability is shallow: it fixes *which objects* the tuple holds, not their contents.

2.4 Conclusion

Use a **tuple** for a fixed-size record whose parts belong together and must not change (coordinates, `(row, col)` keys, RGB values, database rows, multiple return values, constants) and when the value must be hashable. Use a **list** for a growing or changing collection of similar items. Immutability protects against accidental modification, against aliasing bugs where another part of the program changes shared data, and against corrupting dictionary or set keys; it also tells the reader that the value is fixed.

2.5 Alternative concept and complexity analysis

A sparse grid stores only zones that have passengers: `counts[(row, col)] = counts.get((row, col), 0) + 1`, and a missing zone means 0 (`counts.get(zone, 0)`).

Design	Access/update	Space	Measured
List of lists (rows × cols)	O(1) – two index operations	O(rows × cols)	1000 × 1000 grid ≈ 8,064,856 bytes
dict keyed by (row, col)	O(1) average – one hash lookup	O(k), k = non-empty zones	2 non-empty zones ≈ 224 bytes

Both give O(1) average access. The dictionary wins when most zones are empty (a city with thousands of zones but few boarding points); the list wins when most zones are used because

the dictionary costs more bytes per entry and needs hashing, and a list is easier to iterate row by row.

Q3 – Dictionaries: Farmer Helpline Complaint Analysis (7 marks, K4 – Analyse)

3.1 Implementation

Complete the Code – `frequencies` counts words not in the stop set with `dict.get()`; `top` sorts by `(-count, word)` so the highest count comes first and ties are alphabetical.

```
def frequencies(texts, stop):
    counts = {}
    for t in texts:
        for w in t.split():
            if w not in stop:
                counts[w] = counts.get(w, 0) + 1
    return counts

def top(counts, k):
    return sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))[:k]
```

The full program also counts with `setdefault()`, groups complaints by district and by crop (`by_district.setdefault(d, []).append(text)`), uses a set of stop words (O(1) membership), and cleans text with lower-casing and a regular expression that keeps only letters.

q3_dicts.py (lines 1-42)

```python
"""Q3 - Dictionaries: Farmer helpline complaint analysis"""
import random, re, time
from collections import Counter

print("=== Complete the Code (TODO 1 and TODO 2) ===")
calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}

def frequencies(texts, stop):
    counts = {}
    for t in texts:
        for w in t.split():
            if w not in stop:
                counts[w] = counts.get(w, 0) + 1
    return counts

def top(counts, k):
    return sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))[:k]

print(top(frequencies(calls, STOP), 3))

def tokens(text):
    return re.findall(r"[a-z]+", text.lower())

def freq_get(texts, stop):
    c = {}
    for t in texts:
        for w in tokens(t):
            if w not in stop:
                c[w] = c.get(w, 0) + 1
    return c

def freq_setdefault(texts, stop):
    c = {}
    for t in texts:
        for w in tokens(t):
            if w not in stop:
                c.setdefault(w, [0])[0] += 1
```

Screenshot Q3 source code (1 of 3)

q3_dicts.py (lines 43-84)

```
    return {k: v[0] for k, v in c.items()}

def freq_listcount(texts, stop):
    words = [w for t in texts for w in tokens(t) if w not in stop]
    return {w: words.count(w) for w in set(words)}

def freq_counter(texts, stop):
    return Counter(w for t in texts for w in tokens(t) if w not in stop)

print("\n=== Group complaints by district / crop ===")
records = [("Ariyalur", "paddy", "seeds not received"), ("Thanjavur", "paddy",
      "fertiliser price high"),
           ("Ariyalur", "cotton", "pesticide shortage"), ("Salem", "cotton", "subsidy
      delayed"),
           ("Thanjavur", "banana", "seeds delayed")]
by_district, by_crop = {}, {}
for d, crop, text in records:
    by_district.setdefault(d, []).append(text)
    by_crop.setdefault(crop, []).append(text)
print("per district:", {d: len(v) for d, v in by_district.items()})
print("by_district :", by_district)
print("by_crop     :", by_crop)

print("\n=== Edge cases: empty text, case, punctuation ===")
print("empty        :", freq_get([""], STOP))
print("case+punct   :", freq_get(["Seeds, SEEDS! seeds. Price; HIGH?"], STOP))
print("all methods agree:", freq_get(["Seeds, SEEDS! seeds. Price"], STOP) ==
      freq_setdefault(["Seeds, SEEDS! seeds. Price"], STOP) == freq_listcount(["Seeds,
      SEEDS! seeds. Price"], STOP) == dict(freq_counter(["Seeds, SEEDS! seeds. Price"],
      STOP)))

print("\n=== dict views and key order ===")
d = {"b": 2, "a": 1}
d["c"] = 3
print("keys  :", d.keys(), "| values:", d.values(), "| items:", d.items())
ks = d.keys(); d["z"] = 26
print("view is live after insert:", list(ks))

print("\n=== Timing (seconds) - vocabulary grows with the number of complaints (d ~ n/2)
       ===")
random.seed(3)
def alpha(i):
    """letters-only word for index i (the tokenizer keeps only a-z)"""
    s = ""
    while True:
        s = chr(97 + i % 26) + s
        i //= 26
        if i == 0:
```

Screenshot Q3 source code (2 of 3)

q3_dicts.py (lines 85-107)

```
            return "w" + s

def make(n):
    vocab = [alpha(i) for i in range(n // 2)] + ["seeds", "price", "subsidy",
      "fertiliser", "delayed", "received"]
    return [" ".join(random.choice(vocab) for _ in range(8)) for _ in range(n)]
print(f"{'complaints':>10} {'distinct':>9} {'dict.get':>10} {'setdefault':>11}
      {'Counter':>10} {'list.count':>11}")
last_lc = None
for n in (1000, 10000, 100000):
    texts = make(n)
    d_cnt = len(freq_get(texts, STOP))
    row = []
    for f in (freq_get, freq_setdefault, freq_counter, freq_listcount):
        if f is freq_listcount and n == 100000:
            row.append(f"~{last_lc*100:.0f}*")   # O(n*d) with d ~ n/2 -> x100 when n
      grows x10
        else:
            t0 = time.perf_counter(); f(texts, STOP); t = time.perf_counter() - t0
            if f is freq_listcount: last_lc = t
            row.append(f"{t:.4f}")
    print(f"{n:>10} {d_cnt:>9} {row[0]:>10} {row[1]:>11} {row[2]:>10} {row[3]:>11}")
print("* not run (would take minutes): estimated by the O(n*d) rule, 10x more complaints
       -> ~100x more time")
t = make(2000)
assert freq_get(t, STOP) == freq_setdefault(t, STOP) == freq_listcount(t, STOP) ==
      dict(freq_counter(t, STOP))
print("all four methods give identical counts on 2,000 complaints: True")
```

Screenshot Q3 source code (3 of 3)

$ python q3_dicts.py (output part 1)

```
=== Complete the Code (TODO 1 and TODO 2) ===
[('seeds', 3), ('price', 2), ('received', 2)]

=== Group complaints by district / crop ===
per district: {'Ariyalur': 2, 'Thanjavur': 2, 'Salem': 1}
by_district : {'Ariyalur': ['seeds not received', 'pesticide shortage'], 'Thanjavur':
     ['fertiliser price high', 'seeds delayed'], 'Salem': ['subsidy delayed']}
by_crop     : {'paddy': ['seeds not received', 'fertiliser price high'], 'cotton':
     ['pesticide shortage', 'subsidy delayed'], 'banana': ['seeds delayed']}

=== Edge cases: empty text, case, punctuation ===
empty       : {}
case+punct  : {'seeds': 3, 'price': 1}
all methods agree: True

=== dict views and key order ===
keys  : dict_keys(['b', 'a', 'c']) | values: dict_values([2, 1, 3]) | items:
     dict_items([('b', 2), ('a', 1), ('c', 3)])
view is live after insert: ['b', 'a', 'c', 'z']

=== Timing (seconds) - vocabulary grows with the number of complaints (d ~ n/2) ===
complaints  distinct   dict.get  setdefault    Counter  list.count
      1000       506     0.0017      0.0019     0.0016      0.0318
     10000      5006     0.0180      0.0222     0.0172      4.1131
    100000     50006     0.2845      0.3763     0.2651       ~411*
* not run (would take minutes): estimated by the O(n*d) rule, 10x more complaints -> ~100x
     more time
all four methods give identical counts on 2,000 complaints: True
```

Screenshot Q3 execution and output (1 of 1)

The program prints `[('seeds', 3), ('price', 2), ('received', 2)]`, identical to the expected output. **Why `price` comes before `received`:** both occur twice, so the count does not decide.

The sort key `(-count, word)` then compares the words alphabetically, and `'price' < 'received'` because `p` comes before `r`.

3.2 Design and analysis

- **How a dictionary finds a key:** Python computes `hash(key)`, uses it to pick a slot in a hash table, and compares the stored key (identity first, then `==`) at that slot, probing the next slot only on a collision. The table is resized to keep it at most about two-thirds full, so the expected number of probes is a small constant. Lookup is therefore **O(1) on average and does not depend on the number of keys** (worst case O(n) if every key collided).
- **Dictionary versus `list.count()` for each distinct word:** the dictionary loop makes one pass over n words, O(n). `list.count(w)` scans the entire list for every distinct word, so d distinct words cost O(n·d), which becomes quadratic when the vocabulary grows with the data.
- **Key order (Python 3.7+):** dictionaries preserve **insertion order** (a language guarantee; it was an implementation detail in 3.6). Updating an existing key keeps its position; deleting and re-inserting a key moves it to the end. Order of counts is not by frequency, which is why `top()` sorts explicitly.
- **Views:** `keys()`, `values()` and `items()` return **dynamic view objects**, not copies. They reflect later changes (the demo shows a new key `z` appearing in an earlier `keys()` view), take O(1) to create, support iteration and `len`, and `keys()`/`items()` support set operations such as `a.keys() & b.keys()`.

3.3 Experimental investigation

Complaints (8 random words each) were generated for 1,000, 10,000 and 100,000 records; the vocabulary grows with the data (distinct words d ≈ n/2) as it does in real helpline data. All four methods produced identical dictionaries (checked).

Complaints	Distinct words	dict.get (s)	setdefault (s)	Counter (s)	list.count (s)
1000	506	0.0017	0.0019	0.0016	0.0318
10000	5006	0.0180	0.0222	0.0172	4.1131
100000	50006	0.2845	0.3763	0.2651	~411*

* = not executed: with 100,000 complaints `list.count` needs about 4 × 10^{10} comparisons (minutes to hours), so the value is the estimate obtained by multiplying the measured 10,000-complaint time by 100 (10× more data and 10× more distinct words). The dictionary loop, `setdefault` and `Counter` all grow about linearly (×10 data → ×10 time), while `list.count` grows about ×100 per step. `Counter` is about as fast as the hand-written loop (both are O(n)) and gives `most_common(k)` for free.

Tests: empty text → `{}`; `"Seeds, SEEDS! seeds. Price; HIGH?"` → `{'seeds': 3, 'price': 1}` (case folded, punctuation removed, stop word removed).

3.4 Engineering decision

For 100,000 complaints a day I would deploy **`collections.Counter`** (or the equivalent dictionary loop) with a stop-word **set**: O(n) time, about 0.3 s for 100,000 complaints here, O(d) memory, built-in `most_common(k)`, and the least code to maintain. `list.count()` would not finish in a useful time, and the extra cost of `setdefault` over `get` gives no benefit when only counting.

3.5 Alternative concept and complexity analysis

Method	Time (n words, d distinct)	Space	Comment
dict loop with get()	O(n) average	O(d)	One pass
collections.Counter	O(n) average	O(d)	Same complexity, counting loop is in C; most_common(k) is O(d log k)
list.count() per distinct word	O(n · d)	O(n) for the word list + O(d)	Quadratic when d grows with n

Q4 – Sets: Elective Registration Clash Detection (7 marks, K4 – Analyse)

4.1 Implementation

Complete the Code – TODO 1 uses only set operators: intersection `&` for students in both electives, symmetric difference `^` for exactly one, and difference `-` of the union for those in neither.

```
both        = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither     = all_students - (ai_elective | iot_elective)

# Rewritten with set methods
both_m    = ai_elective.intersection(iot_elective)
one_m     = ai_elective.symmetric_difference(iot_elective)
neither_m = all_students.difference(ai_elective.union(iot_elective))
```

q4_sets.py (lines 1-42)

```
"""Q4 - Sets: Elective registration clash detection"""
import random, time

print("=== Complete the Code (TODO 1 - set operators) ===")
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}

both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)

print("clash :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither :", sorted(neither))

print("\n=== Same result using set methods ===")
both_m = ai_elective.intersection(iot_elective)
one_m = ai_elective.symmetric_difference(iot_elective)
neither_m = all_students.difference(ai_elective.union(iot_elective))
print("clash same:", both_m == both, "| exactly one same:", one_m == exactly_one, "|
      neither same:", neither_m == neither)

print("\n=== Other operations ===")
print("union (registered in at least one):", sorted(ai_elective | iot_elective))
print("only AI :", sorted(ai_elective - iot_elective))
print("only IoT:", sorted(iot_elective - ai_elective))

print("\n=== Removing duplicate registrations ===")
raw = ["192311001", "192311004", "192311001", "192311004", "192311007", "192311001"]
print("raw list :", raw, len(raw))
print("as a set :", sorted(set(raw)), len(set(raw)))
print("order-preserving:", list(dict.fromkeys(raw)))

print("\n=== Edge cases ===")
empty = set()
print("empty & AI :", empty & ai_elective, "| empty | AI == AI:", (empty | ai_elective)
      == ai_elective)
print("identical electives -> clash:", len(ai_elective & set(ai_elective)), "exactly
      one:", len(ai_elective ^ set(ai_elective)))
print("neither with both empty:", len(all_students - (empty | empty)), "of",
      len(all_students))

print("\n=== Sets cannot hold lists but can hold tuples ===")
print({(1, 2), (3, 4)})
try:
```

Screenshot Q4 source code (1 of 3)

q4_sets.py (lines 43-84)

```
    {[1, 2]}
except TypeError as e:
    print("TypeError:", e)

print("\n=== Membership test: x in list vs x in set (worst case: last element, 100 lookups) ===")
print(f"{'n':>9} {'list (s)':>12} {'set (s)':>12} {'ratio':>10}")
for n in (10_000, 100_000, 1_000_000):
    L = list(range(n)); S = set(L)
    t0 = time.perf_counter()
    for _ in range(100): (n - 1) in L
    tl = time.perf_counter() - t0
    t0 = time.perf_counter()
    for _ in range(100): (n - 1) in S
    ts = time.perf_counter() - t0
    print(f"{n:>9} {tl:>12.6f} {ts:>12.6f} {tl/ts:>10.0f}x")

print("\n=== Common students: nested loops vs & ===")
def common_nested(a, b):
    out = []
    for x in a:
        for y in b:
            if x == y:
                out.append(x); break
    return out
def common_sorted(a, b):
    a, b = sorted(a), sorted(b)
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] == b[j]: out.append(a[i]); i += 1; j += 1
        elif a[i] < b[j]: i += 1
        else: j += 1
    return out
random.seed(5)
print(f"{'n=m':>8} {'nested':>10} {'sorted+walk':>12} {'set &':>10} {'same?':>6}")
for n in (500, 2000, 8000):
    a = random.sample(range(n * 3), n); b = random.sample(range(n * 3), n)
    t0 = time.perf_counter(); r1 = common_nested(a, b); t1 = time.perf_counter() - t0
    t0 = time.perf_counter(); r2 = common_sorted(a, b); t2 = time.perf_counter() - t0
    t0 = time.perf_counter(); r3 = set(a) & set(b); t3 = time.perf_counter() - t0
    print(f"{n:>8} {t1:>10.5f} {t2:>12.5f} {t3:>10.5f} {set(r1)==set(r2)==r3!s:>6}")

print("\n=== Hand verification on a small case ===")
```

Screenshot Q4 source code (2 of 3)

q4_sets.py (lines 85-89)

```
A, B = {1, 2, 3, 4}, {3, 4, 5, 6}
print("A & B =", A & B, "(hand: {3,4})")
print("A ^ B =", A ^ B, "(hand: {1,2,5,6})")
print("A - B =", A - B, "(hand: {1,2})")
print("A | B =", A | B, "(hand: {1,2,3,4,5,6})")
```

Screenshot Q4 source code (3 of 3)

```
$ python q4_sets.py   (output part 1)

=== Complete the Code (TODO 1 - set operators) ===
clash : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither : ['192311003', '192311005', '192311006', '192311008']

=== Same result using set methods ===
clash same: True | exactly one same: True | neither same: True

=== Other operations ===
union (registered in at least one): ['192311001', '192311002', '192311004', '192311007',
      '192311009', '192311010']
only AI : ['192311001', '192311007']
only IoT: ['192311002', '192311010']

=== Removing duplicate registrations ===
raw list : ['192311001', '192311004', '192311001', '192311004', '192311007', '192311001'] 6
as a set : ['192311001', '192311004', '192311007'] 3
order-preserving: ['192311001', '192311004', '192311007']

=== Edge cases ===
empty & AI : set() | empty | AI == AI: True
identical electives -> clash: 4 exactly one: 0
neither with both empty: 10 of 10

=== Sets cannot hold lists but can hold tuples ===
{(1, 2), (3, 4)}
TypeError: unhashable type: 'list'

=== Membership test: x in list vs x in set (worst case: last element, 100 lookups) ===
        n     list (s)      set (s)      ratio
    10000     0.008988     0.000011        824x
   100000     0.061660     0.000011       5642x
  1000000     0.973104     0.000013      76822x

=== Common students: nested loops vs & ===
     n=m     nested  sorted+walk      set &  same?
     500    0.00216      0.00019    0.00006   True
```

Screenshot Q4 execution and output (1 of 2)

```
$ python q4_sets.py   (output part 2)

    2000    0.03947      0.00081    0.00027   True
    8000    0.59623      0.00405    0.00095   True

=== Hand verification on a small case ===
A & B = {3, 4} (hand: {3,4})
A ^ B = {1, 2, 5, 6} (hand: {1,2,5,6})
A - B = {1, 2} (hand: {1,2})
A | B = {1, 2, 3, 4, 5, 6} (hand: {1,2,3,4,5,6})
```

Screenshot Q4 execution and output (2 of 2)

The three required lines match the expected output exactly, and the method version gives the same three sets (`clash same: True | exactly one same: True | neither same: True`). Duplicate registrations are removed with `set(raw)` (6 entries → 3), or with `dict.fromkeys(raw)` when the first-seen order must be kept. Reports are produced with `sorted(...)`.

4.2 Design and analysis

Venn diagrams (A = AI elective, B = IoT elective, U = all students)

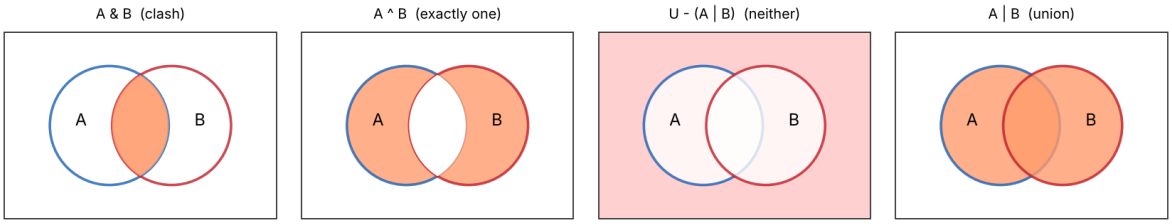


Figure Q4.1 – shaded region is the result of each operation

- **Duplicates:** a set is a hash table of keys. Adding an element whose hash and value already exist finds the existing slot and does nothing, so each value is stored once.
- **Membership:** `x in s` hashes `x` and goes to one slot: O(1) average. `x in list` compares `x` with each item in turn: O(n) (see measurements).
- **Lists versus tuples in sets:** set elements must be hashable. Lists are mutable and unhashable (`TypeError: unhashable type: 'list'`), tuples are hashable if all their items are, so `{(1, 2), (3, 4)}` works.

4.3 Experimental investigation

n register numbers	x in list (s, 100 searches)	x in set (s, 100 searches)	Ratio
10000	0.008988	0.000011	824x
100000	0.061660	0.000011	5642x
1000000	0.973104	0.000013	76822x

The searched value was the last element (worst case for the list). List time grows linearly with n (×10 data → ×10 time) while the set time stays flat (noise level).

n = m	Nested loops (s)	Sorted + walk (s)	set & (s)	Same result
500	0.00216	0.00019	0.00006	True
2000	0.03947	0.00081	0.00027	True
8000	0.59623	0.00405	0.00095	True

Nested loops grow quadratically (×4 data → ×15 time), sorting and walking and `&` grow almost linearly. Edge cases: an empty elective gives an empty intersection and an unchanged union; identical electives give a clash set equal to the elective and an empty exactly-one set; with both electives empty all 10 students are in the “neither” group. **Hand check** with A = {1,2,3,4}, B = {3,4,5,6}: `A&B = {3,4}`, `A^B = {1,2,5,6}`, `A-B = {1,2}`, `A|B = {1,…,6}` – the program printed exactly these.

4.4 Conclusion

Convert a list to a set before processing when you need many membership tests, duplicate removal, or set algebra (common/different elements). What is lost: **order**, **duplicates and counts** (how many times a value occurred), **indexing and slicing**, and the ability to store unhashable items such as lists.

4.5 Alternative concept and complexity analysis

Approach (lists of sizes n and m)	Time	Extra space
Nested loops	O(n · m)	O(1) besides the result
Sort both lists and walk through together	O(n log n + m log m)	O(n + m) for sorted copies (O(1) if sorted in place)
Sets and &	O(n + m) average (build both sets, then iterate over the smaller)	O(n + m)

Q5 – List Comprehension: Cleaning Sensor Readings (6 marks, K3 – Apply)

5.1 Implementation

Valid readings are −50 ≤ °C ≤ 50 (anything outside is a sensor error). Three versions produce the cleaned Celsius list, the Fahrenheit list (`F = C × 9/5 + 32`) and the alarm minutes. **Assumption:** an alarm is a *valid* reading outside 2–8 °C, so sensor errors are not reported as alarms.

```
# comprehension version
clean_c = [c for c in r if -50 <= c <= 50]
clean_f = [round(c * 9 / 5 + 32, 2) for c in clean_c]
alarms  = [m for m, c in enumerate(r) if -50 <= c <= 50 and not (2 <= c <= 8)]

# map/filter version
clean_c = list(filter(lambda c: -50 <= c <= 50, r))
clean_f = list(map(lambda c: round(c * 9 / 5 + 32, 2), clean_c))
```

q5_comprehension.py (lines 1-42)

```python
"""Q5 - List comprehension: cleaning cold-storage sensor readings"""
import random, sys, time

random.seed(11)
def make(n):
    return [round(random.uniform(-60, 60), 1) if random.random() < 0.1 else
      round(random.uniform(1, 10), 1) for _ in range(n)]

def v_loop(r):
    clean_c, clean_f = [], []
    for c in r:
        if -50 <= c <= 50:
            clean_c.append(c)
            clean_f.append(round(c * 9 / 5 + 32, 2))
    alarms = []
    for minute, c in enumerate(r):
        if -50 <= c <= 50 and not (2 <= c <= 8):
            alarms.append(minute)
    return clean_c, clean_f, alarms

def v_comp(r):
    clean_c = [c for c in r if -50 <= c <= 50]
    clean_f = [round(c * 9 / 5 + 32, 2) for c in clean_c]
    alarms = [m for m, c in enumerate(r) if -50 <= c <= 50 and not (2 <= c <= 8)]
    return clean_c, clean_f, alarms

def v_map(r):
    clean_c = list(filter(lambda c: -50 <= c <= 50, r))
    clean_f = list(map(lambda c: round(c * 9 / 5 + 32, 2), clean_c))
    alarms = [m for m, c in filter(lambda p: -50 <= p[1] <= 50 and not (2 <= p[1] <= 8),
       enumerate(r))]
    return clean_c, clean_f, alarms

sample = [4.5, 9.1, 99.0, -70.2, 1.5, 3.0, 8.0, 2.0]
print("=== Demo on", sample, "===")
for f in (v_loop, v_comp, v_map):
    print(f"{f.__name__:7s}", f(sample))
print("all three identical:", v_loop(sample) == v_comp(sample) == v_map(sample))

print("\n=== Dictionary and set comprehensions ===")
print("dict {minute: celsius} of valid readings:", {m: c for m, c in enumerate(sample)
      if -50 <= c <= 50})
print("set of distinct valid readings         :", sorted({c for c in sample if -50 <= c
      <= 50}))
print("dict {celsius: fahrenheit}             :", {c: round(c * 9 / 5 + 32, 2) for c in
      sample if -50 <= c <= 50})

```

Screenshot Q5 source code (1 of 2)

q5_comprehension.py (lines 43-71)

```
print("\n=== Edge cases ===")
allgood = [5.0] * 5
allbad = [100.0, -99.0, 51.0, -50.5]
for name, d in (("all valid  ", allgood), ("all invalid", allbad), ("empty      ", [])):
    print(name, v_comp(d), "| three versions identical:", v_loop(d) == v_comp(d) ==
      v_map(d))

print("\n=== Timing (seconds, best of 3) ===")
print(f"{'n':>9} {'for+append':>11} {'comprehension':>14} {'map/filter':>11}
      {'identical':>10}")
for n in (10_000, 100_000, 1_000_000):
    d = make(n)
    res = []
    for f in (v_loop, v_comp, v_map):
        ts = []
        for _ in range(3):
            t0 = time.perf_counter(); out = f(d); ts.append(time.perf_counter() - t0)
        res.append(min(ts))
    print(f"{n:>9} {res[0]:>11.4f} {res[1]:>14.4f} {res[2]:>11.4f} {str(v_loop(d) ==
      v_comp(d) == v_map(d)):>10}")

print("\n=== Memory: list comprehension vs generator expression (n = 1,000,000) ===")
d = make(1_000_000)
lst = [c for c in d if -50 <= c <= 50]
gen = (c for c in d if -50 <= c <= 50)
print("list comprehension :", sys.getsizeof(lst), "bytes")
print("generator expression:", sys.getsizeof(gen), "bytes")

print("\n=== Generator expression as the alternative: sum / count without building a
      list ===")
t0 = time.perf_counter(); s1 = sum(c for c in d if -50 <= c <= 50); t1 =
      time.perf_counter() - t0
t0 = time.perf_counter(); s2 = sum([c for c in d if -50 <= c <= 50]); t2 =
      time.perf_counter() - t0
print(f"sum(generator)={s1:.1f} in {t1:.4f}s | sum(list)={s2:.1f} in {t2:.4f}s | equal:
      {abs(s1-s2) < 1e-6}")
```

Screenshot Q5 source code (2 of 2)

```
$ python q5_comprehension.py   (output part 1)

=== Demo on [4.5, 9.1, 99.0, -70.2, 1.5, 3.0, 8.0, 2.0] ===
v_loop  ([4.5, 9.1, 1.5, 3.0, 8.0, 2.0], [40.1, 48.38, 34.7, 37.4, 46.4, 35.6], [1, 4])
v_comp  ([4.5, 9.1, 1.5, 3.0, 8.0, 2.0], [40.1, 48.38, 34.7, 37.4, 46.4, 35.6], [1, 4])
v_map   ([4.5, 9.1, 1.5, 3.0, 8.0, 2.0], [40.1, 48.38, 34.7, 37.4, 46.4, 35.6], [1, 4])
all three identical: True

=== Dictionary and set comprehensions ===
dict {minute: celsius} of valid readings: {0: 4.5, 1: 9.1, 4: 1.5, 5: 3.0, 6: 8.0, 7: 2.0}
set of distinct valid readings         : [1.5, 2.0, 3.0, 4.5, 8.0, 9.1]
dict {celsius: fahrenheit}             : {4.5: 40.1, 9.1: 48.38, 1.5: 34.7, 3.0: 37.4, 8.0:
      46.4, 2.0: 35.6}

=== Edge cases ===
all valid   ([5.0, 5.0, 5.0, 5.0, 5.0], [41.0, 41.0, 41.0, 41.0, 41.0], []) | three versions
       identical: True
all invalid ([], [], []) | three versions identical: True
empty       ([], [], []) | three versions identical: True

=== Timing (seconds, best of 3) ===
        n  for+append  comprehension  map/filter  identical
    10000      0.0044         0.0061      0.0075       True
   100000      0.0463         0.0351      0.0460       True
  1000000      0.4923         0.4476      0.5369       True

=== Memory: list comprehension vs generator expression (n = 1,000,000) ===
list comprehension : 8448728 bytes
generator expression: 208 bytes

=== Generator expression as the alternative: sum / count without building a list ===
sum(generator)=4954515.0 in 0.0613s | sum(list)=4954515.0 in 0.0762s | equal: True
```

Screenshot Q5 execution and output (1 of 1)

5.2 Design and analysis

- **General form:** `[expression for item in iterable if condition]`; with a conditional expression: `[a if condition else b for item in iterable]`. Several `for` clauses nest left to right.
- **When a comprehension is harder to read than a loop:** with more than two loops or conditions, long expressions, several outputs, `try/except`, or side effects (printing, appending elsewhere). A loop with named intermediate steps is clearer in those cases.
- **Dictionary and set comprehensions** (shown in the output): `{m: c for m, c in enumerate(r) if -50 <= c <= 50}` maps minute → temperature of valid readings; `{c for c in r if -50 <= c <= 50}` is the set of distinct valid readings.
- **List comprehension versus generator expression:** `[...]` is eager – it builds the whole list in memory; `(...)` is lazy – it yields one item at a time, can be used once and cannot be indexed or sliced.

5.3 Experimental investigation

n readings	for + append (s)	comprehension (s)	map/filter + lambda (s)	Identical
10000	0.0044	0.0061	0.0075	True
100000	0.0463	0.0351	0.0460	True
1000000	0.4923	0.4476	0.5369	True

The three versions gave identical results for every size (and for all-valid, all-invalid and empty inputs). The comprehension is typically the fastest, but only by about 5–15 %; `map`/`filter` with a

`lambda` is no faster because every element pays for a Python function call. Differences at 10,000 readings are within timer noise.

Container (1,000,000 readings, valid values)	sys.getsizeof (bytes)
List comprehension	8,448,728
Generator expression	208

5.4 Conclusion

No – a list comprehension is not always best. It is the best choice when the whole result is needed more than once, or must be indexed or sorted. The measurements show it is only slightly faster than a plain loop, and when the result is consumed once (a sum, a count, `any`, writing to a file) a generator expression produces the same answer in about the same time using **208 bytes instead of about 8 MB**. A loop is better when the logic is complex.

5.5 Alternative concept and complexity analysis

`total = sum(c for c in readings if -50 <= c <= 50)` replaces the list with a generator. Measured: `sum(generator)=4954515.0 in 0.0613s | sum(list)=4954515.0 in 0.0762s | equal: True`.

Approach	Time	Memory
List comprehension then sum()	O(n)	O(n) – the list of valid values
Generator expression inside sum()	O(n)	O(1) – one value at a time

Q6 – Nested Data: A Railway Timetable (7 marks, K4 – Analyse)

6.1 Implementation

`timetable` maps a train number to a dictionary with `name`, `stops` (a list of `(station, arrival, departure)` tuples) and `days` (a set). `trains_at(station, day)` scans the trains, keeps those that run on the day and stop at the station, and sorts them with `sorted(..., key=lambda r: hm(r[3]))`, where `hm` converts `HH:MM` to minutes so the sort is numeric. Five trains were entered and the whole timetable is printed in a neat layout.

q6_timetable.py (lines 1-42)

```
"""Q6 - Nested data: a railway timetable"""
import random, sys, time

DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
def hm(t):                      # "HH:MM" -> minutes, so sorting by time is numeric
    h, m = t.split(":"); return int(h) * 60 + int(m)

timetable = {
    "12601": {"name": "Chennai Mail", "stops": [("Chennai Central", "--", "20:00"),
      ("Katpadi", "22:10", "22:12"), ("Salem", "01:15", "01:20"), ("Coimbatore",
      "04:30", "--")], "days": {"Mon", "Wed", "Fri"}},
    "12605": {"name": "Pallavan Exp", "stops": [("Chennai Egmore", "--", "15:45"),
      ("Villupuram", "18:05", "18:07"), ("Tiruchirappalli", "21:00", "--")], "days":
      set(DAYS)},
    "12635": {"name": "Vaigai Exp", "stops": [("Chennai Egmore", "--", "12:25"),
      ("Villupuram", "14:48", "14:50"), ("Tiruchirappalli", "17:35", "17:40"),
      ("Madurai", "20:30", "--")], "days": {"Tue", "Thu", "Sat"}},
    "16127": {"name": "Guruvayur Exp", "stops": [("Chennai Egmore", "--", "21:30"),
      ("Villupuram", "00:05", "00:07"), ("Tiruchirappalli", "03:20", "03:25"),
      ("Madurai", "06:10", "--")], "days": {"Mon", "Fri", "Sun"}},
    "22651": {"name": "Cheran Exp", "stops": [("Chennai Central", "--", "22:15"),
      ("Katpadi", "00:25", "00:27"), ("Salem", "03:40", "03:45"), ("Coimbatore",
      "06:50", "--")], "days": {"Wed", "Sat"}},
}

def trains_at(tt, station, day):
    out = []
    for no, info in tt.items():
        if day in info["days"]:
            for st, arr, dep in info["stops"]:
                if st == station:
                    out.append((no, info["name"], arr, dep))
    return sorted(out, key=lambda r: hm(r[3] if r[3] != "--" else r[2]))

def print_timetable(tt):
    for no, info in tt.items():
        print(f"{no} {info['name']:15s} runs: {','.join(d for d in DAYS if d in
      info['days'])}")
        for st, arr, dep in info["stops"]:
            print(f"      {st:18s} arr {arr:>5s}  dep {dep:>5s}")

print("=== Timetable ===")
print_timetable(timetable)

print("\n=== Access the third station's departure time of train 12601 ===")
print('timetable["12601"]["stops"][2][2] ->', timetable["12601"]["stops"][2][2])

print("\n=== Trains stopping at Villupuram on Tue, sorted by departure ===")
for r in trains_at(timetable, "Villupuram", "Tue"):
    print(r)
print("\n=== Edge cases ===")
print("unknown station 'Goa'        :", trains_at(timetable, "Goa", "Tue"))
print("Villupuram on Mon (12635 does not run, 12605/16127 do):", [r[0] for r in
      trains_at(timetable, "Villupuram", "Mon")])
```

Screenshot Q6 source code (1 of 3)

q6_timetable.py (lines 43-84)

```
print("Katpadi on Tue (nothing runs):", trains_at(timetable, "Katpadi", "Tue"))

def build_index(tt):
    idx = {}
    for no, info in tt.items():
        for k, (st, arr, dep) in enumerate(info["stops"]):
            idx.setdefault(st, []).append((no, k))
    return idx

def query_indexed(tt, idx, station, day):
    out = []
    for no, k in idx.get(station, []):
        info = tt[no]
        if day in info["days"]:
            st, arr, dep = info["stops"][k]
            out.append((no, info["name"], arr, dep))
    return sorted(out, key=lambda r: hm(r[3] if r[3] != "--" else r[2]))

idx = build_index(timetable)
print("indexed result equals scan result:", query_indexed(timetable, idx, "Villupuram",
      "Tue") == trains_at(timetable, "Villupuram", "Tue"))

def flat_query(flat, station, day):
    out = [(no, nm, arr, dep) for no, nm, st, arr, dep, days in flat if st == station
      and day in days]
    return sorted(out, key=lambda r: hm(r[3] if r[3] != "--" else r[2]))

print("\n=== Generated timetables: query time & index memory ===")
random.seed(8)
stations = [f"S{i:04d}" for i in range(500)]
def gen(n):
    tt = {}
    for t in range(n):
        k = random.randint(5, 12)
        names = random.sample(stations, k)
        base = random.randint(0, 600)
        stops = []
        for j, s in enumerate(names):
            m = base + j * 45
            stops.append((s, f"{(m//60)%24:02d}:{m%60:02d}",
      f"{((m+2)//60)%24:02d}:{(m+2)%60:02d}"))
      tt[f"T{t:05d}"] = {"name": f"Train {t}", "stops": stops, "days":
      set(random.sample(DAYS, random.randint(1, 7)))}
    return tt

print(f"{'trains':>7} {'stops':>8} {'no index (ms)':>14} {'indexed (ms)':>13} {'speed-
      up':>9} {'index bytes':>12} {'same':>5}")
```

Screenshot Q6 source code (2 of 3)

q6_timetable.py (lines 85-97)

```
for n in (100, 1000, 10000):
    tt = gen(n)
    flat = [(no, i["name"], st, a, d, i["days"]) for no, i in tt.items() for st, a, d in
       i["stops"]]
    total = len(flat)
    t0 = time.perf_counter(); idx = build_index(tt); tb = time.perf_counter() - t0
    qs = [(random.choice(stations), random.choice(DAYS)) for _ in range(50)]
    t0 = time.perf_counter(); r1 = [trains_at(tt, s, d) for s, d in qs]; t1 =
      (time.perf_counter() - t0) / 50 * 1000
    t0 = time.perf_counter(); r2 = [query_indexed(tt, idx, s, d) for s, d in qs]; t2 =
      (time.perf_counter() - t0) / 50 * 1000
    mem = sys.getsizeof(idx) + sum(sys.getsizeof(v) for v in idx.values()) +
      sum(sys.getsizeof(e) for v in idx.values() for e in v)
    print(f"{n:>7} {total:>8} {t1:>14.4f} {t2:>13.5f} {t1/t2:>8.0f}x {mem:>12}
      {str(r1==r2):>5}")
    if n == 10000:
        t0 = time.perf_counter(); r3 = [flat_query(flat, s, d) for s, d in qs]; t3 =
      (time.perf_counter() - t0) / 50 * 1000
        print(f"        flat-list scan of all {total} stop tuples: {t3:.4f} ms/query |
      same answers: {r3 == r1} | index build once: {tb*1000:.1f} ms")
```

Screenshot Q6 source code (3 of 3)

$ python q6_timetable.py (output part 1)

```
=== Timetable ===
12601 Chennai Mail    runs: Mon,Wed,Fri
      Chennai Central    arr    --  dep 20:00
      Katpadi            arr 22:10  dep 22:12
      Salem              arr 01:15  dep 01:20
      Coimbatore         arr 04:30  dep    --
12605 Pallavan Exp    runs: Mon,Tue,Wed,Thu,Fri,Sat,Sun
      Chennai Egmore     arr    --  dep 15:45
      Villupuram         arr 18:05  dep 18:07
      Tiruchirappalli    arr 21:00  dep    --
12635 Vaigai Exp      runs: Tue,Thu,Sat
      Chennai Egmore     arr    --  dep 12:25
      Villupuram         arr 14:48  dep 14:50
      Tiruchirappalli    arr 17:35  dep 17:40
      Madurai            arr 20:30  dep    --
16127 Guruvayur Exp   runs: Mon,Fri,Sun
      Chennai Egmore     arr    --  dep 21:30
      Villupuram         arr 00:05  dep 00:07
      Tiruchirappalli    arr 03:20  dep 03:25
      Madurai            arr 06:10  dep    --
22651 Cheran Exp      runs: Wed,Sat
      Chennai Central    arr    --  dep 22:15
      Katpadi            arr 00:25  dep 00:27
      Salem              arr 03:40  dep 03:45
      Coimbatore         arr 06:50  dep    --

=== Access the third station's departure time of train 12601 ===
timetable["12601"]["stops"][2][2] -> 01:20

=== Trains stopping at Villupuram on Tue, sorted by departure ===
('12635', 'Vaigai Exp', '14:48', '14:50')
('12605', 'Pallavan Exp', '18:05', '18:07')

=== Edge cases ===
unknown station 'Goa'        : []
Villupuram on Mon (12635 does not run, 12605/16127 do): ['16127', '12605']
```

Screenshot Q6 execution and output (1 of 2)

```
$ python q6_timetable.py   (output part 2)

Katpadi on Tue (nothing runs): []
indexed result equals scan result: True

=== Generated timetables: query time & index memory ===
 trains    stops  no index (ms)  indexed (ms)  speed-up  index bytes  same
    100      847         0.0253       0.00189       13x        96696  True
   1000     8518         0.2099       0.01224       17x       599536  True
  10000    84604         2.8679       0.14319       20x      5504992  True
       flat-list scan of all 84604 stop tuples: 2.7789 ms/query | same answers: True |
     index build once: 41.5 ms
```

Screenshot Q6 execution and output (2 of 2)

6.2 Design and analysis

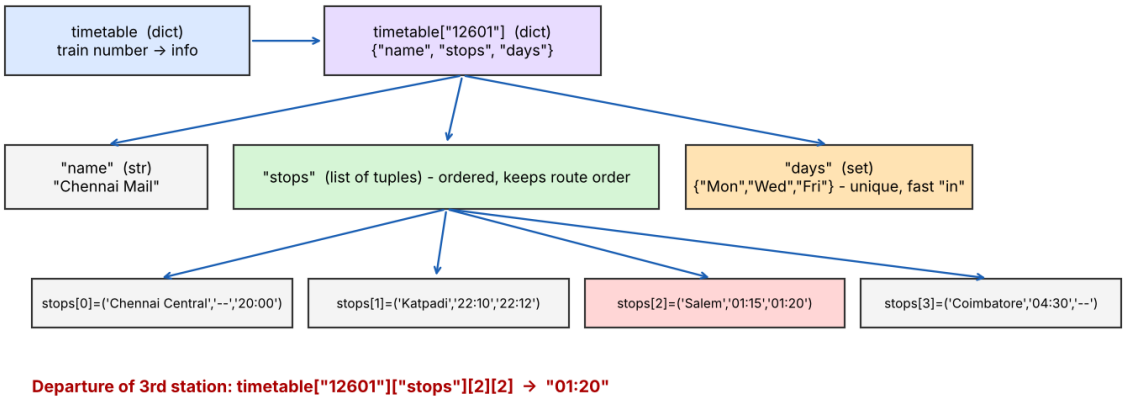


Figure Q6.1 – the nested structure and the access path to a departure time

Expression for the departure time of the third station of train 12601:
`timetable["12601"]["stops"][2][2]` → `"01:20"` (train → `stops` list → third tuple (index 2) → departure field (index 2)).

Why each type is used

- **Dictionary (outer):** unique train numbers, direct lookup by key, O(1) average.
- **Dictionary (inner):** named fields (`name`, `stops`, `days`) – readable access by name.
- **List of stops:** the order of stations matters and a route is read in sequence.
- **Tuple per stop:** a fixed record `(station, arrival, departure)` that should not change by accident.
- **Set of days:** unique values, no order needed, fast `day in days`.

Work for one station query and an index

Without an index the program must look at every train and every stop: with T trains and s stops per train the work is **O(T · s) = O(total stops)** for each query, plus O(k log k) to sort the k matches. **Index design:** a dictionary `station → list of (train number, position in stops)` built once in O(total stops). A query then reads one dictionary entry and visits only the k trains that stop there: O(k + k log k) after an O(1) lookup, instead of O(total stops).

```
def build_index(tt):
    idx = {}
    for no, info in tt.items():
        for k, (st, arr, dep) in enumerate(info["stops"]):
            idx.setdefault(st, []).append((no, k))
    return idx
```

6.3 Experimental investigation

Timetables of 100, 1,000 and 10,000 trains (5–12 stops each, 500 stations, random running days) were generated; each time is the mean of 50 random queries. Results of the indexed and the unindexed query were identical in every case.

Trains	Total stops	No index (ms)	Indexed (ms)	Speed-up	Index memory (bytes)	Same
100	847	0.0253	0.00189	13x	96,696	True
1,000	8,518	0.2099	0.01224	17x	599,536	True
10,000	84,604	2.8679	0.14319	20x	5,504,992	True

The indexed query is an order of magnitude faster at every size. Both columns grow roughly linearly with the number of trains because the number of stations is fixed (500), so the number of trains per station k also grows with T; the gap is smaller than the theoretical factor (about the number of stations) because both versions must create the result tuples and sort them, and the unindexed scan skips the stops of trains that do not run on the chosen day. The index costs extra memory (about 5.5 MB for 10,000 trains) and a one-off build time (41.5 ms).

Tests: an unknown station returns `[]`; a train that does not run on the chosen day is left out (e.g. Villupuram on Monday excludes train 12635, Katpadi on Tuesday returns `[]`).

6.4 Conclusion

An index makes each query faster because it replaces a search through **all** stops by one hash lookup that jumps straight to the stops of the requested station. The trade-off is **extra memory** (one entry per stop), **build time**, and the duty to **keep the index up to date** whenever a train is added, changed or removed (otherwise answers become wrong). It pays off when queries are frequent and the timetable changes rarely.

6.5 Alternative concept and complexity analysis

With one flat list of `(train, station, arrival, departure, days)` tuples every query is a scan: `[r for r in flat if r[1] == station and day in r[4]]`. Measured at 10,000 trains: `flat-list scan of all 84604 stop tuples: 2.7789 ms/query | same answers: True | index build once: 41.5 ms`.

Design	Time per query	Space
Flat list of tuples, scanned	O(total stops)	O(total stops); train name and day-set repeated in every tuple (redundant)
Nested dictionaries, no index	O(total stops)	O(total stops), no repetition of name/days
Nested dictionaries + station index	O(k) lookup + O(k log k) sort, k = trains at the station	O(total stops) + O(total stops) for the index

Q7 – Design a Contact-Merging Program from First Principles (10 marks, K5 – Evaluate)

Stage 1 – Interpret the problem (2 marks)

- **Input:** contacts from two phones and a SIM card; each contact has an optional name (text), a list of phone numbers in any format, and a list of e-mail addresses. Any field can be missing.
- **Output:** a list of unique persons; each has a set of names (original spellings kept), a set of normalised numbers and a set of e-mails; contacts without a usable number are reported separately and never silently dropped.
- **Rule for "same person":** two contacts are the same person if they share at least one **normalised phone number**. Names are *not* the identity: spellings differ ("Dr. Ravi" / "ravi kumar") and different people can share a name. The rule is transitive: if A shares a number with B and B shares another with C, then A, B and C are one person.
- **Data structures:** list (input and output sequence), dictionary (normalised number → person record, O(1) lookup), sets (names, numbers, e-mails: duplicates vanish automatically), string/regex (normalisation).

Measurable requirements:

- R1 – `+91 98400 12345`, `9840012345`, `098400-12345` and `0091 98400 12345` all normalise to `9840012345` (tested).
- R2 – no normalised number appears in two output persons.
- R3 – no information is lost: every input number and e-mail appears in the output.
- R4 – contacts with no usable number are listed as orphans; missing or `None` fields never raise an exception.
- R5 – merging 1,000 contacts takes under 1 s and 100,000 contacts under 5 s.

Stage 2 – Design (3 marks)

- **Number normalisation:** keep digits only; remove the prefixes `0091`, `91` (when 12 digits long) and a leading `0` (when 11 digits long) so Indian numbers become 10 digits; numbers with fewer than 7 digits are rejected as unusable.
- **Name normalisation:** lower-case, keep letters only, remove titles (dr, mr, mrs, ms, prof, sri, smt), collapse spaces. Names are used for display and for checking, not as the key.

Pseudocode

```
FUNCTION merge(contacts)
    people <- empty dictionary          // normalised number -> record
    orphans <- empty list
    FOR EACH c IN contacts
        nums   <- { normalise_number(n) FOR n IN c.numbers } minus {""}
        emails <- { lowercase(trim(e)) FOR e IN c.emails }
        IF nums is empty THEN APPEND c TO orphans; CONTINUE
        found <- [ k IN nums WHERE k IN people ]      // existing records this contact
touches
        IF found is empty THEN record <- new record(names={}, numbers={}, emails={})
        ELSE record <- people[found[0]]
             FOR EACH other IN found[1:]               // contact links two earlier
records
                 IF people[other] is not record THEN
                     union other's names/numbers/emails INTO record
                     FOR EACH n IN other.numbers: people[n] <- record
        ADD c.name TO record.names;  UNION nums, emails INTO record
        FOR EACH n IN nums: people[n] <- record
    RETURN distinct records in people, orphans
```

Stage 3 – Implement and measure (3 marks)

q7_contacts.py (lines 1-42)

```
"""Q7 - Contact merging from first principles"""
import random, re, string, time

# ---------- normalisation ----------
def norm_number(raw):
    """Keep digits only; drop 0 / 91 / 0091 prefixes so every Indian mobile becomes 10 digits."""
    d = re.sub(r"\D", "", raw or "")
    if d.startswith("0091"): d = d[4:]
    elif len(d) == 12 and d.startswith("91"): d = d[2:]
    elif len(d) == 11 and d.startswith("0"): d = d[1:]
    return d if len(d) >= 7 else ""            # too short -> unusable key

TITLES = {"dr", "mr", "mrs", "ms", "prof", "sri", "smt"}
def norm_name(raw):
    words = re.findall(r"[a-z]+", (raw or "").lower())
    return " ".join(w for w in words if w not in TITLES)

def norm_email(raw):
    return (raw or "").strip().lower()

# ---------- merge: O(n) average, keyed by normalised number ----------
def merge(contacts):
    people = {}          # key -> {"names":set, "numbers":set, "emails":set}
    orphans = []         # contacts with no usable number
    for c in contacts:
        nums = {norm_number(n) for n in (c.get("numbers") or [])} - {""}
        emails = {norm_email(e) for e in (c.get("emails") or []) if e}
        name = c.get("name", "")
        if not nums:
            orphans.append(c); continue
        # a contact may carry several numbers: link them all to one record
        keys = [k for k in nums if k in people]
        if keys:
            main = keys[0]
            for other in keys[1:]:
                if people[other] is not people[main]:       # merge two existing records (union-find style)
                    rec = people[other]
                    people[main]["names"] |= rec["names"]; people[main]["numbers"] |= rec["numbers"]; people[main]["emails"] |= rec["emails"]
                    for n in rec["numbers"]: people[n] = people[main]
            rec = people[main]
        else:
            rec = {"names": set(), "numbers": set(), "emails": set()}
```

Screenshot Q7 source code (1 of 4)

q7_contacts.py (lines 43-84)

```
        if name: rec["names"].add(name.strip())
        rec["numbers"] |= nums; rec["emails"] |= emails
        for n in nums: people[n] = rec
    unique = {id(r): r for r in people.values()}.values()
    return list(unique), orphans

# ---------- alternative: compare every pair, O(n^2) ----------
def merge_pairwise(contacts):
    recs = []
    for c in contacts:
        nums = {norm_number(n) for n in (c.get("numbers") or [])} - {""}
        if not nums: continue
        emails = {norm_email(e) for e in (c.get("emails") or []) if e}
        rec = {"names": {c["name"].strip()} if c.get("name") else set(), "numbers":
      set(nums), "emails": emails}
        recs.append(rec)
    merged = True
    while merged:
        merged = False
        for i in range(len(recs)):
            for j in range(i + 1, len(recs)):
                if recs[i]["numbers"] & recs[j]["numbers"]:
                    for k in recs[i]: recs[i][k] |= recs[j][k]
                    del recs[j]; merged = True; break
            if merged: break
    return recs

def show(recs):
    for r in sorted(recs, key=lambda r: sorted(r["numbers"])[0]):
        print(f"  names={sorted(r['names'])} numbers={sorted(r['numbers'])}
      emails={sorted(r['emails'])}")

print("=== Normalisation tests ===")
for raw in ["+91 98400 12345", "9840012345", "098400-12345", "0091 98400 12345",
      "12-34", ""]:
    print(f"  {raw!r:22} -> {norm_number(raw)!r}")
for raw in ["Dr. Ravi", "ravi kumar", "  RAVI   KUMAR ", None]:
    print(f"  {raw!r:22} -> {norm_name(raw)!r}")

print("\n=== Small list merged: expected result worked out by hand ===")
phone1 = [{"name": "Dr. Ravi", "numbers": ["+91 98400 12345"], "emails": []},
          {"name": "Anitha", "numbers": ["9444011111"], "emails": ["anitha@x.com"]}]
phone2 = [{"name": "ravi kumar", "numbers": ["9840012345", "044-2222 3333"], "emails":
      ["Ravi@Mail.com"]},
          {"name": "Anitha S", "numbers": ["+919444011111"], "emails": []},
          {"name": "Meena", "numbers": ["9000000001"], "emails": []}]
```

Screenshot Q7 source code (2 of 4)

q7_contacts.py (lines 85-126)

```
sim = [{"name": "Ravi K", "numbers": ["044 2222 3333"], "emails": []},     # links to Ravi via the landline
       {"name": "No Number", "numbers": [], "emails": ["x@y.com"]},        # missing field
       {"name": "Meena", "numbers": ["09000000001"], "emails": ["meena@z.com"]}]
allc = phone1 + phone2 + sim
print("input contacts:", len(allc))
print("Hand-worked: Ravi (3 names, numbers 9840012345 & 04422223333, email ravi@mail.com);")
print("             Anitha (2 names, number 9444011111, email anitha@x.com);")
print("             Meena (number 9000000001, email meena@z.com); 1 orphan with no number.")
recs, orphans = merge(allc)
print("program output:")
show(recs)
print("  orphans (no usable number):", [o["name"] for o in orphans])
print("  records:", len(recs), "(hand: 3)")
print("  pairwise result identical:", sorted(map(lambda r: sorted(r['numbers']), merge_pairwise(allc))) == sorted(map(lambda r: sorted(r['numbers']), recs)))

print("\n=== Two different people with the same name ===")
same = [{"name": "Kumar", "numbers": ["9111111111"], "emails": []}, {"name": "Kumar", "numbers": ["9222222222"], "emails": []}]
r, _ = merge(same); print("  records:", len(r), "-> kept separate because the numbers differ")

print("\n=== Missing / odd fields ===")
print("  ", merge([{}, {"name": "A"}, {"numbers": ["abc"]}, {"name": "B", "numbers": None}]))

print("\n=== Timing: dictionary merge vs pairwise comparison ===")
random.seed(21)
def synth(n, dup=0.3):
    people = max(1, int(n * (1 - dup)))
    nums = [str(random.randint(6_000_000_000, 9_999_999_999)) for _ in range(people)]
    out = []
    fmts = [lambda s: s, lambda s: "+91 " + s[:5] + " " + s[5:], lambda s: "0" + s, lambda s: "91" + s]
    for i in range(n):
        p = random.randrange(people) if i >= people else i
        out.append({"name": f"Person {p}", "numbers": [random.choice(fmts)(nums[p])], "emails": [f"p{p}@mail.com"] if random.random() < 0.5 else []})
    random.shuffle(out)
    return out
print(f"{'contacts':>9} {'dict merge (s)':>15} {'unique people':>14} {'pairwise (s)':>14}")
t_pair_1000 = None
for n in (1000, 100_000):
    data = synth(n)
    t0 = time.perf_counter(); recs, _ = merge(data); td = time.perf_counter() - t0
    if n == 1000:
        t0 = time.perf_counter(); pr = merge_pairwise(data); t_pair_1000 = time.perf_counter() - t0
        assert len(pr) == len(recs)
```

Screenshot Q7 source code (3 of 4)

q7_contacts.py (lines 127-131)

```
        ptxt = f"{t_pair_1000:.3f}"
    else:
        ptxt = f"~{t_pair_1000 * 10_000:.0f}*"
    print(f"{n:>9} {td:>15.4f} {len(recs):>14} {ptxt:>14}")
print("* not run: pairwise cost grows as n^2, so 100x more contacts -> ~10,000x more time")
```

Screenshot Q7 source code (4 of 4)

```
$ python q7_contacts.py   (output part 1)

=== Normalisation tests ===
  '+91 98400 12345'      -> '9840012345'
  '9840012345'           -> '9840012345'
  '098400-12345'         -> '9840012345'
  '0091 98400 12345'     -> '9840012345'
  '12-34'                -> ''
  ''                     -> ''
  'Dr. Ravi'             -> 'ravi'
  'ravi kumar'           -> 'ravi kumar'
  '  RAVI   KUMAR '      -> 'ravi kumar'
  None                   -> ''

=== Small list merged: expected result worked out by hand ===
input contacts: 8
Hand-worked: Ravi (3 names, numbers 9840012345 & 04422223333, email ravi@mail.com);
             Anitha (2 names, number 9444011111, email anitha@x.com);
             Meena (number 9000000001, email meena@z.com); 1 orphan with no number.
program output:
  names=['Dr. Ravi', 'Ravi K', 'ravi kumar'] numbers=['4422223333', '9840012345']
      emails=['ravi@mail.com']
  names=['Meena'] numbers=['9000000001'] emails=['meena@z.com']
  names=['Anitha', 'Anitha S'] numbers=['9444011111'] emails=['anitha@x.com']
  orphans (no usable number): ['No Number']
  records: 3 (hand: 3)
  pairwise result identical: True

=== Two different people with the same name ===
  records: 2 -> kept separate because the numbers differ

=== Missing / odd fields ===
   ([], [{}, {'name': 'A'}, {'numbers': ['abc']}, {'name': 'B', 'numbers': None}])

=== Timing: dictionary merge vs pairwise comparison ===
 contacts  dict merge (s)  unique people   pairwise (s)
     1000          0.0051            700          4.480
   100000          0.9013          69999        ~44799*
* not run: pairwise cost grows as n^2, so 100x more contacts -> ~10,000x more time
```

Screenshot Q7 execution and output (1 of 1)

Verification by hand. The small test has 8 input contacts. Worked out by hand: *Ravi* = “Dr. Ravi”, “ravi kumar” (same number 9840012345) and “Ravi K” (shares the landline 04422223333) → 1 person with 3 names, 2 numbers, e-mail `ravi@mail.com` (case folded); *Anitha* = “Anitha” and “Anitha S” (`9444011111` and `+919444011111`) → 1 person; *Meena* (`9000000001` and `09000000001`) → 1 person; “No Number” has no number → orphan. Expected: **3 persons and 1 orphan**; the program printed exactly 3 persons with the same names, numbers and e-mails, and 1 orphan. The dictionary result also equals the pairwise result.

Tests: duplicates, different formats, a contact that links two earlier records (the landline), missing fields (`{}`, only a name, `numbers: None`, a number `"abc"`), and two people named “Kumar” with different numbers (kept as 2 persons).

Contacts	Dictionary merge (s)	Unique persons found	Pairwise comparison (s)
1,000	0.0051	700	4.480
100,000	0.9013	69,999	~44799*

Requirement R5 is met: 1,000 contacts merge in a few milliseconds and 100,000 in about a second. * = not executed; the pairwise time for 100,000 contacts is estimated as 10,000 × the measured 1,000-contact time (see Stage 4).

Stage 4 – Defend the design (2 marks)

- **Dictionary versus comparing every pair:** a dictionary lookup finds the existing record for a number in O(1) average, so n contacts need O(n) work. Comparing every pair needs n(n − 1)/2 comparisons = O(n²): for 1,000 contacts that took about 4–5 s in my measurement against a few milliseconds for the dictionary, and 100,000 contacts would need about 5 × 10⁹ comparisons (hours).
- **Two different people with the same name:** names are not used as the key, so “Kumar” with 9111111111 and “Kumar” with 9222222222 stay as two persons (tested). Their names are merged only when a number or a chain of numbers links the contacts.
- **One remaining limitation:** a number shared by two real people (e.g. a family landline or an office number) would wrongly merge them, and one person who has no number in common between their entries (only different names and e-mails) will not be merged. A further step could use e-mail as a second key or ask the user to confirm uncertain merges.

Alternative concept and complexity analysis

The alternative compares every pair of contacts and merges those that share a number (`merge_pairwise` in the code). My baseline restarts its scan after each merge, so it is even slower than the ideal pairwise algorithm; the ideal one still needs O(n²) comparisons.

Approach	Time	Space	Measured
Dictionary keyed by normalised number	O(n) average	O(n) (records + number keys)	1,000: 0.0051 s; 100,000: 0.9013 s
Compare every pair	O(n²) (n(n−1)/2 comparisons)	O(n)	1,000: 4.480 s; 100,000: ~44799* s (estimated)

Summary of evidence

Question	Evidence in this report	Key result
Q1	Code, output, pseudocode, 3 timing tables	Running sum ≈ 3× fewer operations and ≈ 2–3× faster; insert is O(n)
Q2	TODO 1 completed, memory diagram, `id()` proof, size/time table	Tuple and list use the same memory; immutability is shallow
Q3	TODO 1 & 2 completed, timing table	dict/Counter O(n); list.count O(n·d)
Q4	TODO 1 completed (operators and methods), Venn diagrams, timings	Set lookup O(1); set & is orders of magnitude faster than nested loops
Q5	Three versions verified identical, timings, memory	Generator: 208 bytes vs about 8 MB; comprehension only slightly faster
Q6	Nested structure, index, timings and index memory	Index: roughly 15–20× faster queries for a few MB extra memory
Q7	Requirements, pseudocode, hand verification, timings	Dictionary merge O(n) versus O(n^2) pairwise